Requirements Document: AI-Enabled Skills and Contribution Assessor (PoC Phase)

Project Overview

This project aims to create an AI-enabled system leveraging the GitHub Issue API and ChatGPT to summarize intern contributions and classify skills gained per issue in the Hack for LA organization. It adds skill set labels from a predefined CSV taxonomy., and generates resume-ready bullet points.

Functional Requirements (PoC Scope)

User Input and Interaction

A predefined list of contributors will be used (no text entry or user input in the PoC).

Data Processing

- Fetch comments per issue for each user in the predefined list via GitHub Issue API.
    - Script to find people associated with issue, outputs a file with issue numbers
    - Script for the pilot it will be across multiple repos so that captures name of repo AND issue number for that person
    - PoC: Config file that contains any issues and PRs that the list of users was involved with for the website repo
    - Pilot: all the issues for a person across all HfLA repos
        - Examples: PoC will be all the issues for a person on the website repo
          `repo:hackforla/website involves:JasonUranta`
          https://github.com/search?q=repo%3Ahackforla%2Fwebsite+involves%3AJasonUranta&type=issues
          4 issues, 7 PRs
        - Pilot will be all the issues for a person across all HackforLA repos
          `org:hackforla involves:JasonUranta`

https://github.com/search?q=org%3Ahackforla+involves%3AJasonUranta&type=issues
9 issues, 9 PRs

- Extract context from issue overview and action items: as a context and also as a comment - the prompt needs to make this clear
- Submit combined issue context and user comments to ChatGPT.
- Multiple temp files: one for GitHub comments, one for the ChatGPT response
    1) First file: All the comment text.
        - NAME (GitHub handle)
        - We also want the URL of the comment
        - Comment Content
    2) Second File: The ChatGPT prompt
    3) Third file: ChatGPT Response (labels and bullets output from the API)
    4) Fourth file: (log file) Which comments left on the issue (and URLs) and which labels were added to the issue

Output Generation

- **Bullet Points:**
    - Formal style with software development concepts.
    - Use the STAR (Situation, Task, Action, Result) method for clarity.
    - Maximum of 30 words per bullet point.
    - Explicitly state task status (for PoC, all issues are closed).
    - Resume bullet comment will follow a standardized format:
        - Opening message describing the purpose, with a link to more detail about the automation.
        - Bullet points grouped per user, e.g.:

          @username1
          - Resume Bullet 1
          - Resume Bullet 2

          @username2
          - Resume Bullet 1
          - Resume Bullet 2

- **Skill Classification:**
    - Based on provided CSV file (HfLA_skills.csv), structured as comma-separated labels.
    - Multiple skill labels will be assigned per *issue*.
        - The label comment should include (perhaps) placement of cause for label

    - Label comment formatting:
        - Opening message
        - Markdown checklist of suggested skills:

          - [ ] Python
          - [ ] Project Management

- **Ensure API response times and limits from GitHub and OpenAI are manageable.**

Non-Functional Requirements

Performance

- The entire operation per issue per user must complete within X minutes.
    - 1 minute is our initial choice for an upper bound to the process, later experimentation may give a better indication of when we should check on its functioning and look at the error messages
    - This really has to do with the LLM API limitations, not the GitHub Issue API or Github Actions

Reliability and Error Handling

- Errors from GitHub or OpenAI APIs:
    - Publicly post the error code and user-friendly message to the original issue.
    - Log error details to a structured error log file.
    - These error logs will help identify persistent or pattern-based failures.

Security

- No encryption or anonymization required.
- No anticipated privacy concerns.
- GitHub bot authentication
- ChatGPT API account authentication

Deployment and Environment

- No existing deployment infrastructure.

- Leveraged existing true-github-contributors package, modifying it for AI-enabled summarization and
- skill classification tasks.
- Runs daily across all open issues in the Hack for LA organization using a time-triggered script.
- GitHub is used as the database; output files will be written in **CSV and JSON** formats under the /Data folder.
- Schema for these files is TBD but will support:
    - Associated issue ID
    - Skill labels
    - Contributor ID and name
    - Resume bullets

Scalability

- Designed for low concurrent load via sequential batch processing.

Usability

- No GUI; automated backend process using scripts.

Technical Stack and System Notes

- *SOME* of The original JavaScript codebase (true-github-contributors) is being rewritten in **Python** for use on this project, and all development will use Python going forward. We will need to credit the original author in readme.

Build This as a Modular Per-Issue Pipeline

Even though it's per-issue now, **separate each concern** into functions or CLI stages:

Function	Purpose
fetch_github_issue(issue_number)	Collect title, body, comments, PR links

format_prompt(issue_data, taxonomy_csv)	Create structured prompt text
call_chatgpt(prompt)	Make labeled API request
save_output(issue_number, result)	Dump to JSON or plain text

This way, even when it runs per issue, you've prepped the pipeline for **batch orchestration later**, or **hook-based execution in the cloud**.

Data and API Requirements

GitHub API

- Uses GitHub Issue API to fetch comments and post results.
- Requires a bot account with proper collaborator permissions (avoids spam restrictions).
- Uses a **Personal Access Token** (configured to never expire) for authentication.
- Related documentation:
  - Hack for LA Website Wiki
  - HfLA-Website-Admin: How to update Hackforlabot token

OpenAI API

- GPT-4 Turbo or GPT-4o will be used. Premium ChatGPT API key to be acquired.
- OpenAI rate limits (Tier 1):

- **RPM (Requests per Minute):** 500
- **TPM (Tokens per Minute):** 30,000
- These limits will be respected using throttling logic.

- Excessive token use will be avoided through input size control.
- Consider Costs, per model

Audit and Maintenance

- Log all bot activity, including:
  - User mentions
  - Generated content
  - Errors and diagnostics
- Monitor GitHub and OpenAI dashboards for outages.

Future Considerations (Post-PoC / Pilot Phase)

- Accepting manual contributor input via UI or CLI
- Handling ongoing (open) issues
- More granular user notification on resume bullets (e.g., DM vs. group mention)
- Notifications for HITL reviewers (in the issue/Slack/by New Issues)
- Schema definition
- Data storage (temp and perm)
- Analytics for skill acquisition trends across org
- Codebase will reside in GitHub repo: hackforla/ai-skills-assessor.
- **Batch Processing Option (Later)**
  - Easy to add a loop or cron job over your temp files
- **Cloud-Triggered Future (Webhook)**
  - Full per-issue processing is the optimal design

Thinking Ahead: Future Cloud-Trigger Design

If you're going to trigger this on an issues.closed webhook later, then your full flow becomes:

1. Receive webhook with issue_number
2. Fetch data via GitHub API
3. Format & send to ChatGPT
4. Save labels / optionally write back to GitHub
5. Log locally or to a simple dashboard (e.g., static HTML + JSON log)

You can deploy this as a:

- Cloud function (e.g., Google Cloud Functions, AWS Lambda)
- GitHub App webhook server (free-tier on Fly.io, Render.com, Railway.app)
- Local daemon with polling (during dev)

In that future state, **per-issue, end-to-end is exactly what you want**.

This revised document defines the PoC phase of the AI Skills Assessor project. It captures constraints, goals, and structural considerations needed to implement, evaluate, and shape a full-scale pilot.

Action Items Suggested by ChatGPT for the project (in order):

AI Skills and Contribution Assessor – Project Roadmap (Accurate Status)

Phase 1: Proof of Concept (PoC) – 🟢 Current Phase (In Progress)

Goal: Build and validate a per-issue AI labeling pipeline using ChatGPT and GitHub Actions.
Focus: Demonstrate that AI can infer skill labels and generate resume bullets from GitHub issue comments.

1️⃣ Data Collection

- ☑ ~~Fetch issue and comment data using GitHub API (per user, per issue)~~
- ☑ ~~Store issue title, body, comments, and URLs in structured JSON/CSV~~
- ☑ ~~Config file specifies contributor list and repo scope (`website` repo for now)~~

2️⃣ AI Label & Bullet Generation

- ☐ Format input prompt using issue context + user comments
- ☐ Submit prompt to GPT-4 (OpenAI API rate-limited Tier 1)
- ☐ Generate output:
  - Resume bullets (STAR method, ≤ 30 words each)
  - Markdown checklist of skill labels (from `HfLA_skills.csv`)

3️⃣ Workflow Automation

- ☐ Set up GitHub Action trigger (e.g., “Apply Labels”)

- ☐ Fetch issue content → call ChatGPT → post AI-suggested labels back
- ☐ Implement output logging (comment logs, model responses, errors) in JSON

4️⃣ Human-in-the-Loop Review

- ☐ Human verifies or corrects AI labels and logs acceptance/edits
- ☐ Store reviewed outcomes for future prompt tuning

5️⃣ Evaluation & Reporting

- ☐ Generate reports comparing AI vs human labels
- ☐ Measure processing time (target ≤ 1 min per issue)
- ☐ Record GitHub and OpenAI API error patterns

Phase 2: Pilot (Multi-Repo Validation) – 🔲 Not Started

Goal: Extend the workflow across Hack for LA repos and add label-enforcement logic.

- ☐ Aggregate issues across all HfLA repos
- ☐ Add validation rules for required labels
- ☐ Automate runs (via cron or workflow schedules)
- ☐ Define JSON schema for skill-label data
- ☐ Integrate dashboard view for labeled issues

Phase 3: Optimization & Prompt Tuning – 🔲 Not Started

Goal: Improve accuracy and efficiency using PoC logs.

- ☐ Analyze labeling errors and refine prompts
- ☐ Test prompt variants and compare accuracy
- ☐ Optimize token usage and latency
- ☐ Prototype audit dashboard

Phase 4: Automation & Cloud Deployment – 🔲 Future

Goal: Replace manual “Apply Labels” trigger with event-driven workflow.

- ☐ Deploy webhook trigger (`issues.closed` event → AI labeling).
- ☐ Run serverless via Fly.io / Render / AWS Lambda.
- ☐ Notify reviewers via GitHub comment + Slack/Discord.
- ☐ Support open issues and batch orchestration.

Phase 5: Analytics & Continuous Improvement – 🔲 Future

Goal: Extract organizational insights from labeled data.

- ☐ Analyze skill acquisition trends across contributors.
- ☐ Publish metrics (accuracy, coverage, review adjustments).
- ☐ Integrate results into Hack for LA dashboards.

Phase 6: Full Production Rollout – 🔲 Future

Goal: Fully autonomous labeling system with audit oversight.

- [ ] Stable, cost-optimized labeling service.
- [ ] API key rotation + prompt version control.
- [ ] Dashboard integration with recognition metrics.
- [ ] Optional UI for manual override or re-labeling.